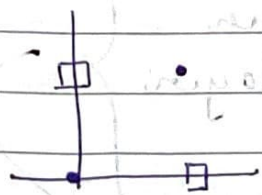


Week-2 DL

L1. Linearly Separable Funcⁿ

- eg. XOR



x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

so set of w 's which satisfies all pts. at

this is not linearly separable simple boolean funcⁿ.

- most real-world data is not linearly separable & always contain outliers.

eg.



with a single perceptron, we cannot handle this.

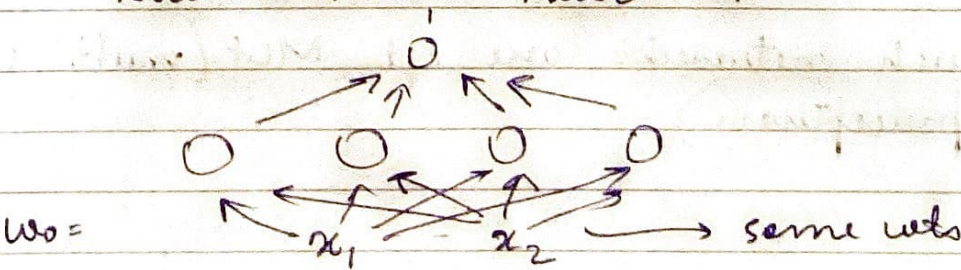
- out of 16 boolean func^{ns} only XOR & !XOR are not linearly separable.

L2

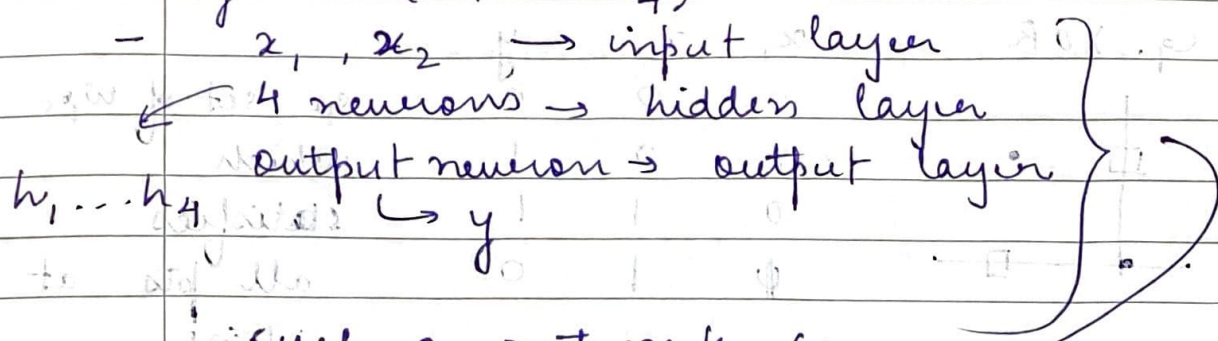
Representaⁿ Power of Network of Perceptron

- any boolean funcⁿ can be done using a network of perceptrons

True = +1 False = -1



- bias (w_0) of -2 means that each perceptron will fire if the wt. sum of the inputs is ≥ 2 . Each is connected to another perceptron by wts. ($w_1 \rightarrow w_4$)



- Such a network can implement any boolean funcⁿ irrespective of (linearly or not linearly separable). We can learn w_1 to w_4 wts. to implement this "funcⁿ".

→ Each layer in hidden layer fires only for a specific inputs (adjust bias appropriately). Set $w_1 \rightarrow w_4$ properly.

- we can use the same idea for ≥ 3 inputs. for 3, there will be 8 perceptrons.
 $n \rightarrow 2^n \rightarrow$ perceptrons

- Any boolean funcⁿ can be represented by 1 hidden layer with 2^n perceptrons and 1 output layer with 1 perceptron.

$n \uparrow$, no. of perceptrons $\uparrow$ exp.

- Such networks are cld MLP (multi layered perceptron)

L3

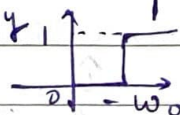
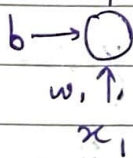
Sigmoid Neuron

$y = f(x)$, $x \in \mathbb{R}^n$ (and not only $\{0, 1\}^n$)
 $y \in \mathbb{R}^n$ ("not" $\{0, 1\}$)
 → arbitrary funcⁿ

we want a network to represent approx. such funcⁿ $\Rightarrow$ intuition behind UAT.

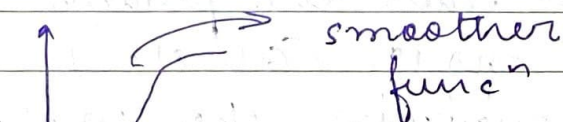
Perceptrons $\rightarrow$ Sigmoid Neurons

— this Thresholding logic isn't good.
 because the funcⁿ is step funcⁿ.



we want sth. smoother than this.

$$z = \sum_{i=1}^n w_i x_i$$



smoother funcⁿ

log funcⁿ $\Rightarrow$ $y = \frac{1}{1 + e^{-z}}$
 (sigmoid funcⁿ family)

$$z = -w_0 + \sum_{i=1}^n w_i x_i = -w_0 + w^T x$$

$$\text{when } w^T x \rightarrow \infty, y \rightarrow 1$$

$$w^T x \rightarrow -\infty, y \rightarrow 0$$

Now, y can have any real value betw. 0 & 1 (probabilities). We won't have a sharp transition around w_0 .

smooth, continuous, differentiable

Perceptron is not differentiable at $-w_0$.

24 Supervised ML Setup

- σ (sigmoid neuron)
- goal of perceptron is to just find a linear boundary, but for not linearly separable it will be hard to achieve 0 error.
- data $\Rightarrow \{x_i, y_i\}_{i=1}^n$
n data pts., & m features $y = f(x)$
there exists some funcⁿ, but we don't know that.

model $\Rightarrow$ over approx. btw. x & y .

$$\hat{y} = \frac{1}{1 + e^{-w^T x}}, \quad \hat{y} = w^T x, \quad \hat{y} = x^T w x$$

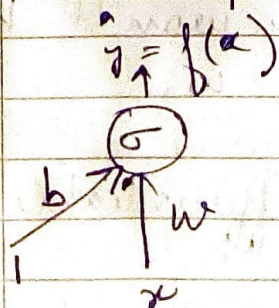
learning algo $\Rightarrow$ learn the params w of the model (GD, etc.)

obj. funcⁿ $\Rightarrow$ minimize the error
$$L(w) = \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$\min_w L(w)$$

find w 's which minimizes the above funcⁿ

25 Learning Params: Guess Work



$$f(x) = \frac{1}{1 + e^{-(wx + b)}}$$

N pairs of $(x, y) \rightarrow$ input data

training obj. $\leftarrow \min_{(w, b) \in R} L(w, b) = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$

- If we find w^*, b^* s.t.
 $f(x_1) \rightarrow y_1, \quad f(x_2) \rightarrow y_2$
 in other words, the pts. must lie on the sigmoid.
- Random guessing to make sure to decrease the loss towards 0.
 $\hookrightarrow$ this process becomes too brute force & can't be done for many data pts. dataset.

LG Taylor Series : Learning Params

- reach min. value quickly w/o brute force

$R^2 \left\{ \begin{array}{l} \theta = [w, b] \\ \Delta \theta = [\Delta w, \Delta b] \end{array} \right\}$ at each step, the loss decreases hopefully

$\theta_{\text{new}} = \theta + \overset{\text{small value}}{\eta \cdot \Delta \theta} \rightarrow$ update rule

$\hookrightarrow$ this comes from Taylor series

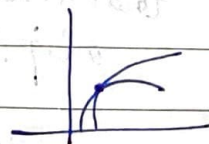
approximates any continuously differentiable funcⁿ $L(w)$ with degree n .

Better the degree, better the approximation.

$$n=1; L(w) = L(w_0) + \frac{L'(w_0)}{1!} (w - w_0)$$



$$n=2; L(w) = L(w_0) + \frac{L'(w_0)}{1!} (w - w_0) + \frac{L''(w_0)}{2!} (w - w_0)^2$$



- it works in a small neighbourhood only. (works similarly for the 2D funcⁿ as well)

in a plane

L7: Gradient Learning Params

$$\Delta \theta = u$$

$$L(\theta + \eta u) = L(\theta) + \eta u^T \nabla_{\theta} L(\theta) + \frac{\eta^2}{2!} u^T \nabla_{\theta}^2 L(\theta) u$$

this is the gradient

$$\text{eg. } y = w^2 + b^2$$

$$\frac{\partial y}{\partial w} = 2w$$

$$\frac{\partial y}{\partial b} = 2b$$

$$\begin{bmatrix} 2w \\ 2b \end{bmatrix} \rightarrow \nabla_{\theta} L(\theta)$$

partial derivatives

collecⁿ of partials is known as gradient

$$\frac{d^2 y}{dx^2} = 2 \quad \text{gradient of gradient} \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

$$\nabla_{\theta}^2 L(\theta)$$

Hessian matrix

$$\begin{bmatrix} \frac{\partial^2 L}{\partial w^2} & \frac{\partial^2 L}{\partial w \partial b} \\ \frac{\partial^2 L}{\partial w \partial b} & \frac{\partial^2 L}{\partial b^2} \end{bmatrix}$$

- the more η is favourable only if -

$$L(\theta + \eta u) - L(\theta) < 0 \quad \left(\begin{array}{l} \text{new} \\ \text{loss} \end{array} < \begin{array}{l} \text{prev.} \\ \text{loss} \end{array} \right)$$

$$u^T \nabla_{\theta} L(\theta) < 0 \quad \left(\text{that means the gradient would be -ve} \right)$$

- Range $u^T \nabla_{\theta} L(\theta)$

$$-1 \leq \cos \beta = \frac{u^T \nabla_{\theta} L(\theta)}{\|u\| \|\nabla_{\theta} L(\theta)\|} \leq 1$$

means opp. in the direction of the gradient
(u is exactly opp. to gradient vector)

18 GD - Wt. Update Rule

- representaⁿ for how ~~the~~ land at a local minima with gradient as guidance.
- obs. $\rightarrow \eta$ was large leading to crossing of minima. So maybe η should be small.
 $\hookrightarrow$ what is effect of initializaⁿ
- GD $\rightarrow$ dirⁿ u should be 180° w.r.t gradient

$$w_{t+1} = w_t - \eta \nabla w_t \quad \nabla w_t = \frac{\partial L(w, b)}{\partial w}$$

$$b_{t+1} = b_t - \eta \nabla b_t$$

$$\nabla b_t = \frac{\partial L(w, b)}{\partial b}$$

we will do this, until we hit the maximum no. of iteraⁿ

$$L(w, b) = \frac{1}{2} \times (f(x) - y)^2$$

$$\begin{aligned} \nabla w &= \frac{\partial L(w, b)}{\partial w} = \frac{\partial}{\partial w} \left[\frac{1}{2} \times (f(x) - y)^2 \right] \\ &= (f(x) - y) \times \frac{\partial}{\partial w} (f(x)) \end{aligned}$$

$$\nabla w = \sum_{i=1}^n (f(x_i) - y_i) \times f(x_i) \times (1 - f(x_i)) \times$$

$$\nabla b = \sum_{i=1}^n (f(x_i) - y_i) \times f(x_i) \times (1 - f(x_i))$$

- atleast as compared to the brute force method, we never make mistakes. We just reduce our loss slowly.

L9 Representaⁿ of ML Network of σ_s : 1D Funcⁿ

- ML network of σ neurons can be used to approx. any continuous funcⁿ to any desired precision.

$$f(x) : \mathbb{R}^n \rightarrow \mathbb{R}^m$$

→ output $g(x)$

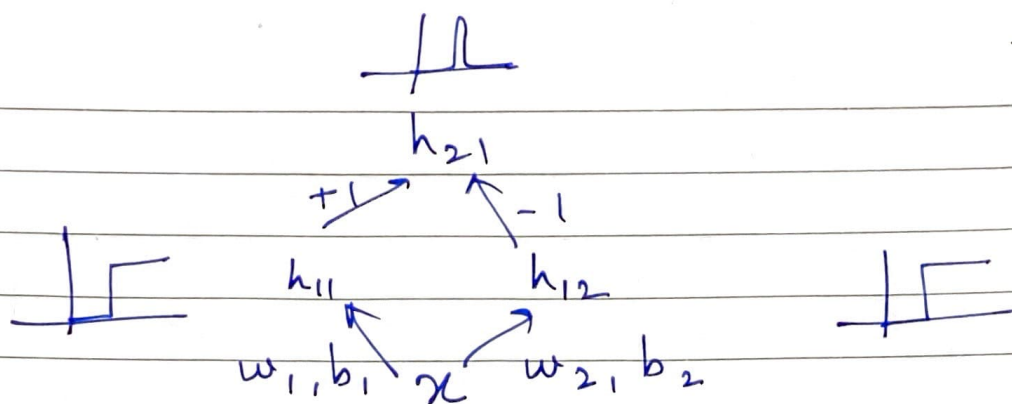
with 1 hidden layer containing enough no. of neurons

$$|g(x) - f(x)| < \epsilon$$

if we have 2 layers, then no. of neurons for each layer can be decreased.

any arbitrary funcⁿ can be approx. using several tower funcⁿ.

DAT



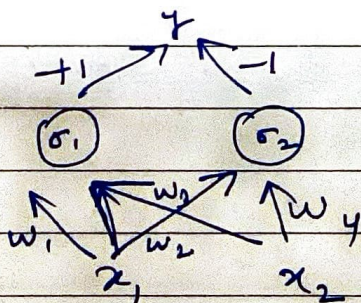
neural network that gives me a tower funcⁿ.

L10

2D Funcⁿ

$$f(x_1, x_2) = \frac{1}{e^{-(w_1 x_1 + w_2 x_2 + b)}}$$

- we can create tower funcⁿ appropriately



sigmoid for
2D funcⁿ